15,471,951 members Sign in

articles Q&A forums stuff lounge ? Search for articles, questions,

Simple Menus That Display Icons - Minimalistic Approach



Alex Cohn

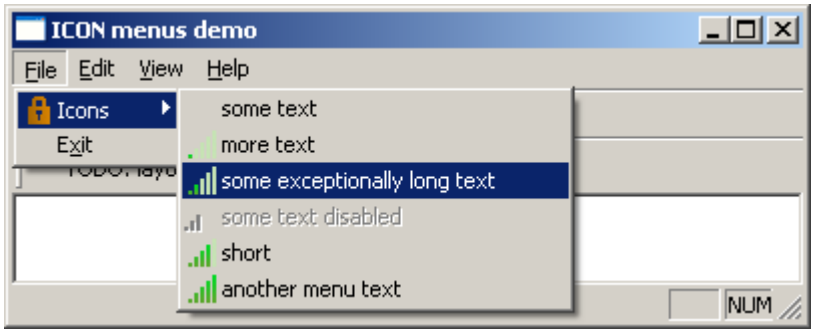
20 Jan 2008 CPOL

Rate me: 4.37/5 (17 votes)

A super-modest approach to owner-drawn menus.

Download demo project - 29.4 KB

Download demo VC6 app - 30.2 KB



Introduction

Most of the wonderful articles on CodeProject that deal with pictures on menus require deep understanding, lots of custom code, and render an application that is completely dependent on the new classes. Our attempt was to generate a simple copy-paste structure for which all the user input is performed with natural tools, i.e., the resource editor embedded into the Microsoft Visual Studio.

The code has been built with Visual Studio 2005 and Visual C++ 6, and tested on Windows XP and Windows 2000.

What you need to build your MFC application

First, make sure that your application works correctly. Then, add the following three functions to your `CMainFrame` class (or to the dialog, if your application is dialog-based):

C++

```
afx_msg void OnDrawItem(int nIDCtl, LPDRAWITEMSTRUCT lpdis);
afx_msg void OnInitMenuPopup(CMenu* pMenu, UINT nIndex, BOOL bSysMenu);
afx_msg void OnMeasureItem(int nIDCtl, LPMEASUREITEMSTRUCT lpmis);
       HMENU GetIconForItem(UINT itemID) const;
```

Add the message map entries for these:

C++

```
ON_WM_DRAWITEM()
ON_WM_MEASUREITEM()
ON_WM_INITMENUPOPUP()
```

somewhere between `BEGIN_MESSAGE_MAP(CMainFrame, CFrameWnd)` and `END_MESSAGE_MAP()`.

Finally, paste these four functions into your CPP file:

C++ Shrink ▲

```
HICON CMainFrame::GetIconForItem(UINT itemID) const
{
    HICON hIcon = (HICON)0;

    if (IS_INTRESOURCE(itemID))
    {
        hIcon = (HICON)::LoadImage(::AfxGetResourceHandle(),
                MAKEINTRESOURCE(itemID), IMAGE_ICON, 0, 0,
                LR_DEFAULTCOLOR | LR_SHARED);
    }

    if (!hIcon)
    {
        CString sItem; // look for a named item in resources

        GetMenu()->GetMenuString(itemID, sItem, MF_BYCOMMAND);
        sItem.Replace(_T(' '), _T('_'));
        // cannot have resource items with space in name

        if (!sItem.IsEmpty())
            hIcon = (HICON)::LoadImage(::AfxGetResourceHandle(), sItem,
                     IMAGE_ICON, 0, 0, LR_DEFAULTCOLOR | LR_SHARED);
    }
    return hIcon;
}
```

```
void CMainFrame::OnDrawItem(int nIDCtl, LPDRAWITEMSTRUCT lpdis)
{
    if ((lpdis==NULL)||(lpdis->CtlType != ODT_MENU))
    {
        CFrameWnd::OnDrawItem(nIDCtl, lpdis);
        return; //not for a menu
    }

    HICON hIcon = GetIconForItem(lpdis->itemID);
    if (hIcon)
    {
        ICONINFO iconinfo;
        ::GetIconInfo(hIcon, &iconinfo);

        BITMAP bitmap;
        ::GetObject(iconinfo.hbmColor, sizeof(bitmap), &bitmap);
        ::DeleteObject(iconinfo.hbmColor);
        ::DeleteObject(iconinfo.hbmMask);

        ::DrawIconEx(lpdis->hDC, lpdis->rcItem.left, lpdis->rcItem.top,
                     hIcon, bitmap.bmWidth, bitmap.bmHeight, 0, NULL, DI_NORMAL);
//      ::DestroyIcon(hIcon); // we use LR_SHARED instead
    }
}

void CMainFrame::OnInitMenuPopup(CMenu* pMenu, UINT nIndex, BOOL bSysMenu)
{
    AfxTrace(_T(__FUNCTION__) _T(": %#0x\n"), pMenu->GetSafeHmenu());
    CFrameWnd::OnInitMenuPopup(pMenu, nIndex, bSysMenu);

    if (bSysMenu)
    {
        pMenu = GetSystemMenu(FALSE);
    }
    MENUINFO mnfo;
    mnfo.cbSize = sizeof(mnfo);
    mnfo.fMask = MIM_STYLE;
    mnfo.dwStyle = MNS_CHECKORBMP | MNS_AUTODISMISS;
    pMenu->SetMenuInfo(&mnfo);

    MENUITEMINFO minfo;
    minfo.cbSize = sizeof(minfo);

    for (UINT pos=0; pos < pMenu->GetMenuItemCount(); pos++)
    {
        minfo.fMask = MIIM_FTYPE | MIIM_ID;
        pMenu->GetMenuItemInfo(pos, &minfo, TRUE);

        HICON hIcon = GetIconForItem(minfo.wID);

        if (hIcon && !(minfo.fType & MFT_OWNERDRAW))
        {
            AfxTrace(_T("replace for \"%s\" id=%u width=%d\n"),
                    (LPCTSTR)sItem, (WORD)minfo.wID, 0); // size.cx);

            minfo.fMask = MIIM_FTYPE | MIIM_BITMAP;
            minfo.hbmpItem = HBMMENU_CALLBACK;
```

```
            minfo.fType = MFT_STRING;

            pMenu->SetMenuItemInfo(pos, &minfo, TRUE);
        }
        else
            AfxTrace(_T("keep for %s id=%u\n"), (LPCTSTR)sItem, (WORD)minfo.wID);
//        ::DestroyIcon(hIcon); // we use LR_SHARED instead
    }
}

void CMainFrame::OnMeasureItem(int nIDCtl, LPMEASUREITEMSTRUCT lpmis)
{
    if ((lpmis==NULL)||(lpmis->CtlType != ODT_MENU))
    {
        CFrameWnd::OnMeasureItem(nIDCtl, lpmis); //not for a menu
        return;
    }

    lpmis->itemWidth = 16;
    lpmis->itemHeight = 16;

    CString sItem;
    GetMenu()->GetMenuString(lpmis->itemID, sItem, MF_BYCOMMAND);

    HICON hIcon = GetIconForItem(lpmis->itemID);

    if (hIcon)
    {
        ICONINFO iconinfo;
        ::GetIconInfo(hIcon, &iconinfo);

        BITMAP bitmap;
        ::GetObject(iconinfo.hbmColor, sizeof(bitmap), &bitmap);
        ::DeleteObject(iconinfo.hbmColor);
        ::DeleteObject(iconinfo.hbmMask);

        lpmis->itemWidth = bitmap.bmWidth;
        lpmis->itemHeight = bitmap.bmHeight;

        AfxTrace(_T(__FUNCTION__) _T(": %d \"%s\"%dx%d ==> %dx%d\n"),
                (WORD)lpmis->itemID, (LPCTSTR)sItem, bitmap.bmWidth,
                bitmap.bmHeight, lpmis->itemWidth, lpmis->itemHeight);
    }
}
```

Now, you can compile your application and see that nothing has changed. To add images next to some menu items, as the ones you see on the screenshot, you simply add icons to your resources. The icon *ID* should be the same as the menu *ID*. That's all.

It is the icon's responsibility to render itself nicely. The size doesn't matter. A good icon editor *(I use Paint.NET with the icon plug-in)* can create icons of arbitrary size and color.

Sometimes, this is not enough. Unfortunately, the menu items that contain sub-menus do not have menu *ID*s. Or at least, you cannot set such an *ID* with the Resource Editor. For these cases, you can

add an icon whose *name* corresponds to the name of the sub-menu. Like this:

C++

```
ICONS                   ICON                    "res\\lock.ico"
...
IDR_MAINFRAME MENU
BEGIN
    POPUP "&File"
    BEGIN
        POPUP "Icons"
```

The method that maps the menu text to the icon uses an underscore (_) to replace the space character; also, note that you can use the **&** character in an icon identifier, but there is a catch: the Windows Explorer will recognize such an icon as the first in the list and use it to represent your executable. The workaround: set the identifier `&` (one character) for the icon you used to call `IDR_MAINFRAME`.

Some of the magic disclosed

We scan the menus as they are to be displayed, and add a flag that the item bitmap should be owner-drawn. If the resource file provides an icon for this item, the bitmap is extracted from the icon. The `WM_MEASUREITEM` message only asks for the size of the bitmap.

Note that all styles like grayed, default, etc. are still available. Unfortunately, the gray icon is displayed in full color when the item is highlighted (selected). You will need a special function (published in the comments by **b ga**) to override this behavior.

We set the menu style to `MNS_CHECKORBMP` purely for aesthetic reasons. But, if some of these items with attached icons are checked, the check mark will override the custom drawing callback. On the other hand, the presented approach may be easily generalized to display custom colorful check signs.

Some words about menu bars

The technique presented here does work with the menu bar (i.e., the part that is always visible above the client area of your window, **File Edit View Help**), but the result is less than perfect (e.g., the underscore is drawn over the image), and requires messing up with the rectangle that Windows provides to the `OnDrawItem` function. Anyway, the attached code (in the zipped demo) draws an icon in the menu bar, too.

What if you have 32-bit (XP-style) icons and Win2K?

It might sound funny, but only recently, I faced a requirement to display true-color 32-bit icons on Windows 2000. On Windows XP, all you have to do is call `DrawIconEx(hdc, left, top, hIcon, width, height, 0, NULL, DI_NORMAL);`. On Windows 2K, though, the alpha channel is ignored by this API. Follows the snippet that is compatible with the older Windows. Note that 32-bit icons are an easy way to represent bitmaps with an alpha channel. Icon format is by no ways limited to the predefined square sizes, and is actually better supported than 32-bit BMP format. I personally use the ICO plug-in for Paint.NET to generate such resources.

Shrink ▲

```
static inline unsigned int alphaBlend(const unsigned int bg, const unsigned int src)
{
    unsigned int    a = src >> 24;    // sourceColor alpha

    // If source pixel is transparent, just return the background
    if (0 == a) return bg;
    if (255 == a) return src;

    // alpha-blend the src and bg colors
    unsigned int rb = (((src & 0x00ff00ff) * a) +
          ((bg & 0x00ff00ff) * (0xff - a))) & 0xff00ff00;
    unsigned int    g  = (((src & 0x0000ff00) * a) +
          ((bg & 0x0000ff00) * (0xff - a))) & 0x00ff0000;

    return (src & 0xff000000) | ((rb | g) >> 8);
}

void MyDrawIcon(HDC hdc, int iconID, int left=0, int top=0, int width=0, int height=0)
{
    if (iconID <= 0)
        return;

    HICON hIcon = LoadIcon(iconID);

    if (!hIcon)
    {
#ifdef _DEBUG
        static bool once = true;
        if (once)
        {
            once = false;
            char str[100];
            HWND hwnd = WindowFromDC(hdc);
            if (GetDlgCtrlID(hwnd))
            {
                sprintf_s(str, "iconID=%d is unknown for control=%d",
                          iconID, GetDlgCtrlID(hwnd));
                MessageBoxA(GetParent(hwnd), str, "Debug", MB_OK | MB_APPLMODAL);
            }
            else
            {
                sprintf_s(str, "iconID=%d is unknown for window=%#x", iconID, hwnd);
                MessageBoxA(hwnd, str, "Debug", MB_OK | MB_APPLMODAL);
            }
        }
```

```
#endif
        return;
    }

#if 1 // WIN2K
    ICONINFO iconInfo;
    GetIconInfo(hIcon, &iconInfo);
    if (iconInfo.hbmMask)
    {
        BITMAP bm;
        GetObject(iconInfo.hbmMask, sizeof(bm), &bm);
        DeleteBitmap(iconInfo.hbmMask);
    }

    if (!iconInfo.hbmColor)
    {
#ifdef _DEBUG
        static bool once = true;
        if (once)
        {
            once = false;
            char str[100];
            HWND hwnd = WindowFromDC(hdc);
            if (GetDlgCtrlID(hwnd))
            {
                sprintf_s(str, "iconInfo.hbmColor is NULL for control=%d",
                          GetDlgCtrlID(hwnd));
                MessageBoxA(GetParent(hwnd), str, "Debug", MB_OK | MB_APPLMODAL);
            }
            else
            {
                sprintf_s(str, "iconInfo.hbmColorhbmColor is NULL for window=%#x", hwnd);
                MessageBoxA(hwnd, str, "Debug", MB_OK | MB_APPLMODAL);
            }
        }
#endif
        return;
    }

    BITMAP bm;
    GetObject(iconInfo.hbmColor, sizeof(bm), &bm);

    if (width == 0)
        width = bm.bmWidth;

    if (height == 0)
        height = bm.bmHeight;

    if (bm.bmBitsPixel != 32)
    {
#ifdef _DEBUG
        static bool once = true;
        if (once)
        {
            once = false;
            char str[100];
            HWND hwnd = WindowFromDC(hdc);
```

```
            if (GetDlgCtrlID(hwnd))
            {
                sprintf_s(str, "iconInfo.hbmColor Bits/Pixel=%d" +
                          " is not correct for control=%d",
                          bm.bmBitsPixel, GetDlgCtrlID(hwnd));
                MessageBoxA(GetParent(hwnd), str, "Debug", MB_OK | MB_APPLMODAL);
            }
            else
            {
                sprintf_s(str, "iconInfo.hbmColor Bits/Pixel=%d" +
                          " is not correct for window=%#x", bm.bmBitsPixel, hwnd);
                MessageBoxA(hwnd, str, "Debug", MB_OK | MB_APPLMODAL);
            }
        }
        DeleteBitmap(iconInfo.hbmColor);
#endif
        return;
    }

    BITMAPINFO bmi = { sizeof(BITMAPINFOHEADER) };
    // get bitmap info
    GetDIBits(hdc, iconInfo.hbmColor, 0, bm.bmHeight, NULL, &bmi, DIB_RGB_COLORS);
    // prepare pixel buffer; note we use 32 bits per pixel
    LPDWORD iconBits = (LPDWORD)malloc(bmi.bmiHeader.biSizeImage);
    // get pixels
    GetDIBits(hdc, iconInfo.hbmColor, 0, bm.bmHeight, iconBits, &bmi, DIB_RGB_COLORS);

    // if width and height are specified, use these for destination bitmap
    bmi.bmiHeader.biWidth = width;
    bmi.bmiHeader.biHeight = height;

    HDC hdcMem = CreateCompatibleDC(hdc);
    LPDWORD pBitsDest = NULL;
    HBITMAP hBmpDest = CreateDIBSection(hdcMem, &bmi, DIB_RGB_COLORS,
                                        (void **)&pBitsDest, NULL, 0);
    HBITMAP hOld = SelectBitmap(hdcMem, hBmpDest);

    // copy the background to memory DC; the pBitsDest buffer will reflect the change
    HWND hwnd = WindowFromDC(hdc);
    if (IsWindow(hwnd) && GetDlgCtrlID(hwnd)) // this is a dialog child
    {
        RECT rc;
        GetWindowRect(hwnd, &rc);
        ScreenToClient(GetParent(hwnd), (LPPOINT)&rc);
        HDC parentDC = GetDC(GetParent(hwnd));
        BitBlt(hdcMem, 0, 0, width, height, parentDC, rc.left+left, rc.top+top, SRCCOPY);
        ReleaseDC(GetParent(hwnd), parentDC);
    }
    else
    {
        BitBlt(hdcMem, 0, 0, width, height, hdc, left, top, SRCCOPY);
    }

    // tile the alpha mask image if the size does not fit
    for (int y=0, ys=0; y < height; y++, (++ys < bm.bmHeight) || (ys = 0))
    {
        for (int x=0, xs=0; x < width; x++, (++xs < bm.bmWidth) || (xs = 0))
```

```
        {
            *pBitsDest = alphaBlend(*pBitsDest, iconBits[xs + ys*bm.bmWidth]);
            pBitsDest++;
        }
    }

    // the bitmap has changed, select it and draw it
    SelectBitmap(hdcMem, hBmpDest);
    BitBlt(hdc, left, top, width, height, hdcMem, 0, 0, SRCCOPY);

    SelectBitmap(hdcMem, hOld);
    DeleteDC(hdcMem);
    DeleteBitmap(iconInfo.hbmColor);
    DeleteBitmap(hBmpDest);
    free(iconBits);

#else
    DrawIconEx(hdc, left, top, hIcon, width, height, 0, NULL, DI_NORMAL);
#endif
}
```

Acknowledgements and updates

Thanks to all commenters, and especially to **Gernot Frisch**, to **b ga**, and to **DarkWeaver5455** and **Joe Partridge** for code review. Please note the comment where **b ga** shows how an icon can be custom drawn to reflect the highlighted or disabled state.

The update of 2/25/2007 resolves the resources leak, pointed to by **Joe Partridge**. The demo project Zip file was updated to reflect the code published in the article. It compiles in VC6, too (these changes are not reflected in the body of the article).

The latest update (1/21/2008) shows how 32-bit icons can be displayed on Windows 2000.

License

This article, along with any associated source code and files, is licensed under The Code Project Open License (CPOL)

Written By

Alex Cohn

Software Developer (Senior) Samsung Telecom Research Israel

Israel

This member has not yet provided a Biography. Assume it's interesting and varied, and probably something to do with programming.

Comments and Discussions

You must Sign In to use this message board.

Search Comments

First Prev Next

Changes Required for VC 6
Member 4187934 23-Nov-10 17:15

I want to bang my head on a wall
mimosa 22-Jun-10 14:31

Re: I want to bang my head on a wall
Alex Cohn 24-Jun-10 5:23

Re: I want to bang my head on a wall
MordachaiWolf 24-Feb-12 3:49

Resource Leak
Joe Partridge 24-Feb-07 20:03

Re: Resource Leak
Alex Cohn 25-Feb-07 3:13

Re: Resource Leak
DarkWeaver5455 25-Feb-07 10:01

Re: Resource Leak
Alex Cohn 25-Feb-07 10:50

Great article - some formatting errors [modified]
DarkWeaver5455 19-Feb-07 21:34

Re: Great article - some formatting errors
Alex Cohn 20-Feb-07 2:57

Re: Great article - some formatting errors
b ga 24-Feb-07 7:45

Latest Update
.dan.g. **14-Feb-07 11:42**

Re: Latest Update
Alex Cohn 14-Feb-07 21:18

Disabled icons for disabled items
b ga **14-Feb-07 7:29**

Re: Disabled icons for disabled items
Alex Cohn 14-Feb-07 20:30

Re: Disabled icons for disabled items
b ga 15-Feb-07 2:55

Re: Disabled icons for disabled items
Alex Cohn 15-Feb-07 3:46

Re: Disabled icons for disabled items [modified]
b ga 15-Feb-07 6:36

Re: Disabled icons for disabled items
Alex Cohn 15-Feb-07 10:22

Re: Disabled icons for disabled items
b ga 15-Feb-07 12:53

Re: Disabled icons for disabled items
Alex Cohn 17-Feb-07 2:38

Re: Disabled icons for disabled items
b ga 18-Feb-07 8:44

Re: Disabled icons for disabled items
Alex Cohn 19-Feb-07 2:41

Re: Disabled icons for disabled items
b ga 19-Feb-07 4:39

Re: Disabled icons for disabled items
Gunn317 29-Nov-07 10:26

Refresh 1 2 3 Next ▷

General News Suggestion Question Bug Answer Joke Praise Rant Admin

Use Ctrl+Left/Right to switch messages, Ctrl+Up/Down to switch threads, Ctrl+Shift+Left/Right to switch pages.

Permalink Layout: fixed | fluid Article Copyright 2006 by Alex Cohn

Advertise
Privacy
Cookies
Terms of Use

Everything else Copyright © CodeProject, 1999-2022

Web04 2.8:2022-10-01:1